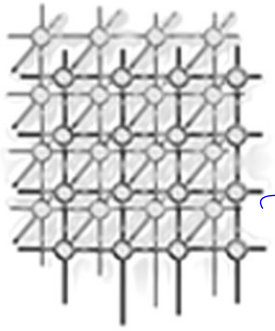
CONCURRENCY AND COMPUTATION: PRACTICE AND EXPERIENCE
*Concurrency Computat.: Pract. Exper.* 2003; **15**:803–820 (DOI: 10.1002/cpe.728)

# The LINPACK Benchmark: past, present and future



Jack J. Dongarra[1,*,†], Piotr Luszczek[1] and Antoine Petitet[2]

[1] *University of Tennessee, Department of Computer Science, Knoxville, TN 37996-3450, U.S.A.*
[2] *Sun Microsystems, Inc., Paris, France*

**SUMMARY**

**This paper describes the LINPACK Benchmark and some of its variations commonly used to assess the performance of computer systems. Aside from the LINPACK Benchmark suite, the TOP500 and the HPL codes are presented. The latter is frequently used to obtained results for TOP500 submissions. Information is also given on how to interpret the results of the benchmark and how the results fit into the performance evaluation process. Copyright © 2003 John Wiley & Sons, Ltd.**

KEY WORDS: benchmarking; BLAS; high-performance computing; HPL; linear algebra; LINPACK; TOP500

## 1. INTRODUCTION

The original LINPACK Benchmark [1] is, in some sense, an accident. It was originally designed to assist users of the LINPACK package [2] by providing information on the execution times required to solve a system of linear equations. The first 'LINPACK Benchmark' report appeared as an appendix in the LINPACK Users' Guide in 1979 [2]. The appendix comprised of data for one commonly used path in the LINPACK software package. Results were provided for a matrix problem of size 100, on a collection of widely used computers (23 computers in all). This was done so users could estimate the time required to solve their matrix problem by extrapolation.

Over the years additional performance data was added, more as a hobby than anything else, and today the collection includes over 1300 different computer systems. In addition to the increasing number of computers, the scope of the benchmark has also expanded. The benchmark report describes the

*Correspondence to: Jack J. Dongarra, University of Tennessee, Department of Computer Science, Knoxville, TN 37996-3450, U.S.A.
†E-mail: dongarra@cs.utk.edu

Contract/grant sponsor: Office of Energy Research, U.S. Department of Energy; contract/grant number: DE-AC05-96OR22464
Contract/grant sponsor: University of Tennessee

*Received 24 December 2001*
*Revised 2 July 2002*

Copyright © 2003 John Wiley & Sons, Ltd.



Table I. Overview of nomenclature and rules for the LINPACK suite of benchmarks. The LINPACK 1000 Benchmark is also known as Toward Peak Performance (TPP) or Best Effort. The Highly-Parallel LINPACK (HPLinpack) Benchmark is also known as the $N \times N$ LINPACK Benchmark or High Parallel Computing (HPC).

| Benchmark name | Matrix dimension | Optimizations allowed | Parallel processing |
|---|---|---|---|
| LINPACK 100 | 100 | Compiler | Compiler parallelization possible |
| LINPACK 1000 | 1000 | Manual | Multiprocessor implementations allowed |
| LINPACK Parallel | 1000 | Manual | Yes |
| HPLinpack | Arbitrary | Manual | Yes |

performance for solving a general dense matrix problem $Ax = b$ in 64-bit floating-point arithmetic at three levels of problem size and optimization opportunity: $100 \times 100$ problem (inner loop optimization), $1000 \times 1000$ problem (three loop optimization—the whole program) and a scalable parallel problem. The names and rules for running the LINPACK suite of benchmarks is given in Table I.

## 2. THE LINPACK PACKAGE AND ORIGINAL LINPACK BENCHMARK

The LINPACK package is a collection of Fortran subroutines for solving various systems of linear equations. The software in LINPACK is based on a decompositional approach to numerical linear algebra. The general idea is the following. Given a problem involving a matrix, $A$, one factors or decomposes $A$ into a product of simple, well-structured matrices which can be easily manipulated to solve the original problem. The package has the capability of handling many different matrix and data types and provides a range of options.

The LINPACK package was based on another package, called the Level 1 Basic Linear Algebra Subroutines (BLAS) [3]. Most of the floating-point work within the LINPACK algorithms is carried out by BLAS, which makes it possible to take advantage of special computer hardware without having to modify the underlying algorithm.

In the LINPACK Benchmark, a matrix of size 100 was originally used because of memory limitations with the computers that were in use in 1979. Such a matrix has 10 000 floating-point elements and could have been accommodated in most environments of that time. At the time it represented a *large enough* problem.

The algorithm used in the timings is based on LU decomposition with partial pivoting. The matrix type is real, general and dense, with matrix elements randomly distributed between $-1$ and 1. The random number generator used in the benchmark is not sophisticated; rather its major attribute is its compactness.

Solving a system of equations requires $O(n^3)$ floating-point operations, more specifically, $2/3n^3 + 2n^2 + O(n)$ floating-point additions and multiplications. Thus, the time (time$_n$) required to solve such problems on a given machine can be approximated with the LINPACK number (time$_{100}$) by

Copyright © 2003 John Wiley & Sons, Ltd.


15320634, 2003, 9, Downloaded from https://onlinelibrary.wiley.com/doi/10.1002/cpe.728 by University Of Maryland, Wiley Online Library on [01/11/2023]. See the Terms and Conditions (https://onlinelibrary.wiley.com/terms-and-conditions) on Wiley Online Library for rules of use; OA articles are governed by the applicable Creative Commons License

Table II. Double precision operation counts for LINPACK 100's DGEFA routine.

| Operation type | Operation counts |
|---|---|
| Addition | 328 350 |
| Multiplication | 333 300 |
| Reciprocal | 99 |
| Absolute value | 5364 |
| Comparison | 4950 |
| Comparison with zero | 5247 |

the following extrapolation formula:

$$\text{time}_n = \frac{\text{time}_{100} \cdot n^3}{100^3}$$

Operation counts for the most computationally intensive routine of the benchmark are given in Table II. The table shows that even for a small matrix (of order 100), multiplications and additions dominate the total operation count. The extrapolation formula is also useful because, on most modern CPUs, floating-point multiplications and additions take (almost) the same number of cycles.

## 3. PERFORMANCE CHARACTERIZATION AND IMPROVEMENT

### 3.1. Concepts

The performance of a computer is a complicated issue, a function of many interrelated quantities. These quantities include the application, the algorithm, the size of the problem, the high-level language, the implementation, the human level of effort used to optimize the program, the compiler's ability to optimize, the age of the compiler, the operating system, the architecture of the computer and the hardware characteristics. The results presented for benchmark suites should not be extolled as measures of total system performance (unless enough analysis has been performed to indicate a reliable correlation of the benchmarks to the workload of interest) but, rather, as reference points for further evaluations.

From this point onwards, by performance we mean the number of millions of floating point operations per second often measured in terms of megaflops (Mflop $s^{-1}$). In the context of the LINPACK benchmark, gigaflops (Gflop $s^{-1}$) are also used as the number of billions of floating point operations per second. It is customary to include both additions and multiplications in the count of Mflop $s^{-1}$, and the operands are assumed to be 64-bit floating-point values.

The manufacturer usually refers to peak performance when describing a system. This peak performance is arrived at by counting the number of floating-point additions and multiplications that can be completed in a period of time, usually the cycle time of the machine. For example, an Intel

Copyright © 2003 John Wiley & Sons, Ltd.


15320634, 2003, 9, Downloaded from https://onlinelibrary.wiley.com/doi/10.1002/cpe.728 by University Of Maryland, Wiley Online Library on [01/11/2023]. See the Terms and Conditions (https://onlinelibrary.wiley.com/terms-and-conditions) on Wiley Online Library for rules of use; OA articles are governed by the applicable Creative Commons License



Table III. Theoretical peak and LINPACK 100 performance numbers of various CPUs.

| Machine | Cycle time (MHz) | Peak Performance (Mflop $s^{-1}$) | LINPACK 100 Performance (Mflop $s^{-1}$) | System efficiency (%) |
|---|---|---|---|---|
| Intel Pentium III | 750 | 750 | 138 | 18.4 |
| Intel Pentium 4 | 2530 | 5060 | 1190 | 23.5 |
| Intel Itanium | 800 | 3200 | 600 | 18.5 |
| AMD Athlon | 1200 | 2400 | 557 | 23.3 |
| Compaq Alpha | 500 | 1000 | 440 | 44.0 |
| IBM RS/6000 | 450 | 1800 | 503 | 27.9 |
| NEC SX-5 | 250 | 8000 | 856 | 10.7 |
| Cray SV-1 | 300 | 1200 | 549 | 45.7 |

Pentium III with a cycle time of 750 MHz has two floating point units: an adder and multiplier. During each cycle the results of either the adder or multiplier can be completed, and thus the peak performance is:

$$R_{\text{peak}} = \frac{1 \text{ operation}}{1 \text{ cycle}} \times 750 \text{ MHz} = 750 \text{ Mflop s}^{-1}$$

Table III shows the peak performance for a number of high-performance computers. We treat the peak theoretical performance as a limit that is guaranteed by the manufacturer not to be exceeded by programs—a sort of *speed of light* for a given computer. The LINPACK Benchmark illustrates this point quite well. In practice, as Table III shows, there may be a significant difference between peak theoretical and actual performance [4]. We are not claiming that Table III reflects the overall performance of a given system. On the contrary, we believe that no single number ever can. It does, however, reflect the performance of a dedicated machine for solving a dense system of linear equations. Since the dense matrix problem is very regular, the performance achieved is quite high, possibly still too high for some common applications to achieve and to be characterized by. However, LINPACK numbers give a good correction of peak performance.

In the following sections, we focus on performance-improving techniques which are relevant to the LINPACK benchmark: *loop unrolling* and *data reuse*.

### 3.2. Loop unrolling

It is a frequently-observed fact that the bulk of the central processor time for a program is localized in 3% or less of the source code [5]. Often the critical code (from a timing perspective) consists of one or more short inner loops typified, for instance, by the scalar product of two vectors. On scalar computers, simple techniques for optimizing such loops should then be most welcome. Loop unrolling (a generalization of *loop doubling*) applied selectively to time-consuming loops is one such technique [6,7]. When a loop is unrolled, its contents are replicated one or more times, with appropriate adjustments to array indices and loop increments. Loop unrolling enhances performance,

Copyright © 2003 John Wiley & Sons, Ltd.


15320634, 2003, 9, Downloaded from https://onlinelibrary.wiley.com/doi/10.1002/cpe.728 by University Of Maryland, Wiley Online Library on [01/11/2023]. See the Terms and Conditions (https://onlinelibrary.wiley.com/terms-and-conditions) on Wiley Online Library for rules of use; OA articles are governed by the applicable Creative Commons License

because there is the direct reduction in loop overhead (the increment, test and branch function). For advanced computer architectures (employing segmented or pipe-lined functional units), the greater density of non-overhead operations permits higher levels of concurrency within a particular segmented unit (e.g. unrolling could allow more than one multiplication to be concurrently active on a segmented machine such as the IBM Power processor). Furthermore, unrolling often increases concurrency between independent functional units on computers so equipped or those with fused multiple-add instructions (the IBM Power processor, which has independent multiplier and adder units, could obtain concurrency between addition for one element and multiplication for the following element). However, on machines with vector instructions, the unrolling technique has the opposite effect. Compilers would always try to detect vector operations in the loop, but the unrolling inhibits this and the resulting vector code might become scalar; consequently the performance degrades.

### 3.3. Vector operations

To observe data reuse patterns we examine the algorithm used in LINPACK and look at how the data are referenced. We see that, at each step of the factorization process, vector operations are performed that modify a full submatrix of data. This update causes a block of data to be read, updated and written back to central memory. The number of floating-point operations is $2/3n^3$ and the number of data references, both loads and stores, is $2/3n^3$. Thus, for every add/multiply pair we must perform a load and store of the elements, unfortunately obtaining little reuse of data. Even though the operations are fully vectorized, there is a significant bottleneck in data movement, resulting in poor performance. On vector computers this translates into two vector operations and three vector-memory references, usually limiting the performance to well below peak rates. On super-scalar computers this results in a large amount of data movement and updates. To achieve high-performance rates, this *operation-to-memory-reference rate* must be higher.

In some sense, this is the problem with doing simple vector operations on a vector or super-scalar machine. The bottleneck is in moving data and the rate of execution is limited by this quantity. We can see this by examining the rate of data transfers and the peak performance. Level 1 BLAS only operates on vectors. The implemented algorithms tend to do more data movement than is necessary. As a result, the performance of the routines in LINPACK suffers on high-performance computers where data movement is as costly as floating-point operations. Today's computer architectures usually have multiple stages in the memory hierarchy as shown in Figure 1. One can gain high performance by restructuring algorithms to exploit this hierarchical organization. To come close to gaining peak performance, one must optimize the use of the lowest level of memory (i.e. retain information as long as possible before the next access to lower level of memory hierarchy), obtaining as much reuse as possible.

### 3.4. Matrix–vector operations

One approach to restructuring algorithms to exploit hierarchical memory involves expressing the algorithms in terms of matrix–vector operations. These operations have the benefit that they can reuse data and achieve a higher rate of execution than the vector counterpart. In fact, the number of floating-point operations remains the same; only the data reference pattern is changed. This change results in a operation-to-memory-reference rate on vector computers of effectively two vector floating-point

Copyright © 2003 John Wiley & Sons, Ltd.


15320634, 2003, 9, Downloaded from https://onlinelibrary.wiley.com/doi/10.1002/cpe.728 by University Of Maryland, Wiley Online Library on [01/11/2023]. See the Terms and Conditions (https://onlinelibrary.wiley.com/terms-and-conditions) on Wiley Online Library for rules of use; OA articles are governed by the applicable Creative Commons License

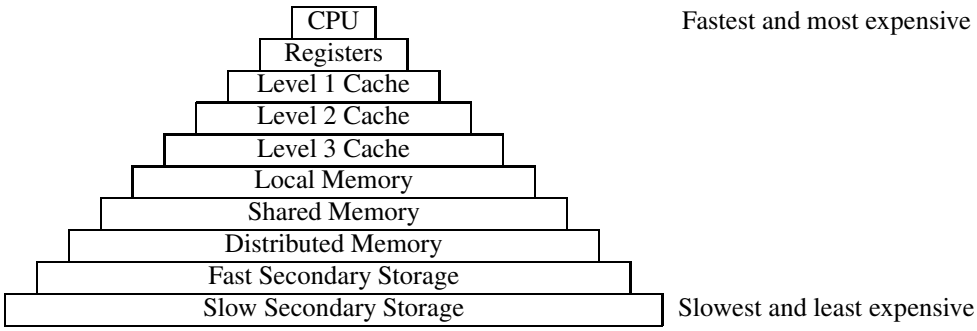



Figure 1. Computer storage hierarchy.

operations and one vector-memory reference. Level 2 BLAS [8] was proposed in order to support the development of software that would be both portable and efficient across a wide range of machine architectures, with emphasis on vector-processing machines. Many of the frequently used algorithms of numerical linear algebra can be coded so that the bulk of the computation is performed by calls to Level 2 BLAS routines; efficiency can then be obtained by utilizing tailored implementations of the Level 2 BLAS routines. On vector-processing machines, one of the aims of such implementations is to keep the vector lengths as long as possible and in most algorithms the results are computed one vector (row or column) at a time. In addition, on vector-register machines performance is increased by reusing the results of a vector register and not storing the vector back into memory.

Unfortunately, this approach to software construction is often not well suited to computers with a hierarchy of memory and true parallel-processing computers. For those architectures, it is often preferable to partition the matrix or matrices into blocks and to perform the computation by matrix–matrix operations on the blocks [8–10]. By organizing the computation in this fashion we provide for full reuse of data while the block is held in the cache or local memory. This approach avoids excessive movement of data to and from memory and gives a *surface-to-volume* effect for the ratio of operations to data movement. In addition, on architectures that provide for parallel processing, parallelism can be exploited in two ways:

Do they 1D Surface-to-Volume?

1. operations on distinct blocks may be performed in parallel; and
2. within the operations on each block, scalar or vector operations may be performed in parallel.

### 3.5. Matrix–matrix operations

To accommodate the portability of matrix–matrix operations, a set of Level 3 BLAS routines have been developed, targeted at the matrix–matrix operations [12]. If the vectors and matrices involved are of order $n$, then the original BLAS (Level 1) includes operations that are of order $O(n)$, the extended or Level 2 BLAS provides operations of order $O(n^2)$, and the latest BLAS provides

Copyright © 2003 John Wiley & Sons, Ltd.


15320634, 2003, 9, Downloaded from https://onlinelibrary.wiley.com/doi/10.1002/cpe.728 by University Of Maryland, Wiley Online Library on [01/11/2023]. See the Terms and Conditions (https://onlinelibrary.wiley.com/terms-and-conditions) on Wiley Online Library for rules of use; OA articles are governed by the applicable Creative Commons License

operations of order $O(n^3)$ (hence the use of the term Level 3 BLAS). There is a long history of block algorithms: early algorithms utilized a small main memory, with tape or disk as secondary storage [12–17]. More recently, several researchers have demonstrated the effectiveness of block algorithms on a variety of modern computer architectures with vector-processing or parallel-processing capabilities [9,10,13,17–22]. Additionally, full blocks (and hence the multiplication of full matrices) might appear as a subproblem when handling large sparse systems of equations [14,23–25]. Finally, it has been shown that matrix–matrix operations can be exploited further. LU factorization (the method of choice for the LINPACK Benchmark code) can be formulated recursively [28]. The recursive formulation achieves better performance [29] than a block algorithm [30]. This is due to the lower memory traffic of the recursive method, which is achieved through better utilization of Level 3 BLAS. The result carries on to the case of out-of-core computations [31]. Interestingly, the recursive algorithm cannot be implemented in standard Fortran 77 (due to the lack of recursive functions and subroutines), unless explicit code for handling the frame stack is provided [28] or an explicit calculation of the update sequence is performed. In Fortran 90, on the other hand, implementation requires careful consideration of the runtime data copying process, which may significantly decrease the performance. A much more elegant implementation may be easily achieved in C.

## 4. LINPACK BENCHMARK SUITE EVOLUTION

Over recent years, the LINPACK Benchmark has evolved from a simple listing for one matrix problem to an expanded benchmark describing the performance at three levels of problem size on several hundred computers. The benchmark today is used by scientists worldwide to evaluate computer performance, particularly for innovative advanced-architecture machines.

As mentioned earlier, performance is a complex issue. To accommodate its evaluation, the LINPACK Benchmark suite provides three separate benchmarks that can be used to evaluate computer performance on a dense system of linear equations: the first for a $100 \times 100$ matrix, the second for a $1000 \times 1000$ matrix. The third benchmark, in particular, is dependent on the algorithm chosen by the manufacturer and the amount of memory available on the computer being benchmarked. For details refer to Table I.

In the case of LINPACK 100, the problem size was relatively small and no changes were allowed to the LINPACK software. Moreover, no explicit attempt was made to use special hardware features or to exploit the vector capabilities or multiple processors. The compilers on some machines may, of course, generate optimized code that itself accesses special features. Thus, as described before, many high-performance machines may not have reached their asymptotic execution rates. However, the benchmark is still important because it approximates the performance rates of numerically intensive codes written by the user and optimized by an optimizing compiler quite well.

The fact that a vendor-supplied code could achieve much higher performance rates than any compiler-optimized code is reflected in the LINPACK Benchmark suite by LINPACK 1000. To begin with, the problem size is larger (matrix of order 1000). In addition, modifying (or even replacing) the algorithm and software is permitted to achieve as high an execution rate as possible. Thus, the hardware has more opportunity for reaching so called *near-asymptotic rates*. Figure 2 illustrates the concept of the asymptotic rate: BLAS routines exhibit higher performance rates as the matrix size increases and algorithm set-up overhead becomes negligible. However, at a certain point the performance rate

Copyright © 2003 John Wiley & Sons, Ltd.


15320634, 2003, 9, Downloaded from https://onlinelibrary.wiley.com/doi/10.1002/cpe.728 by University Of Maryland, Wiley Online Library on [01/11/2023]. See the Terms and Conditions (https://onlinelibrary.wiley.com/terms-and-conditions) on Wiley Online Library for rules of use; OA articles are governed by the applicable Creative Commons License

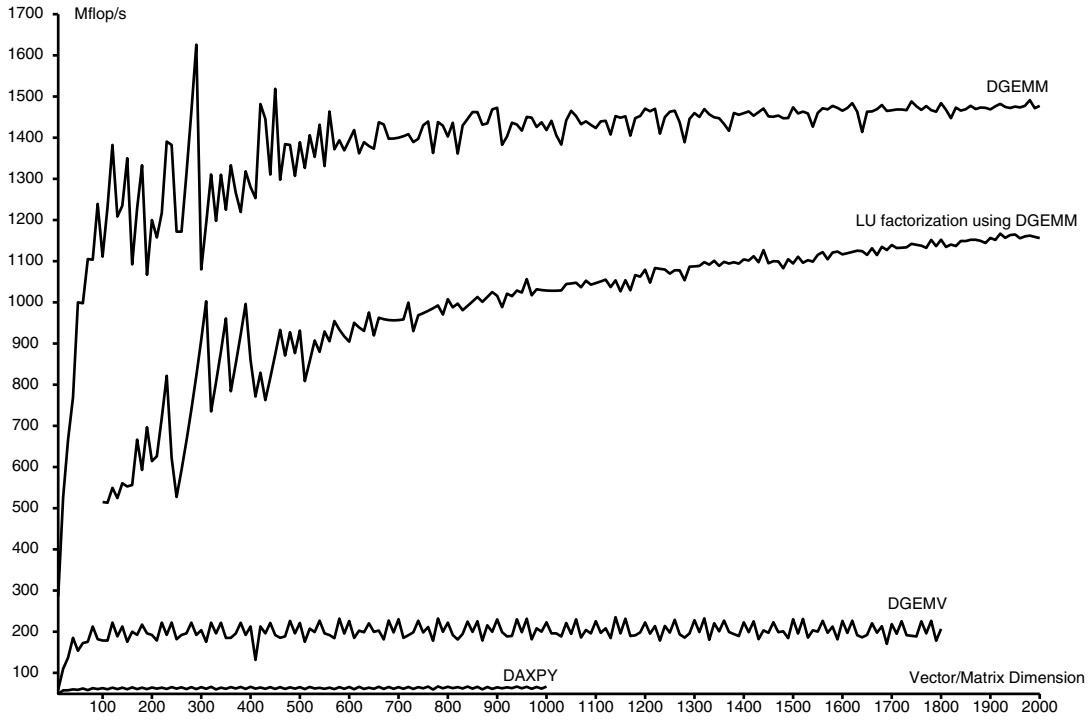



Figure 2. Asymptotic performance rate of BLAS routines `DAXPY` (Level 1), `DGEMV` (Level 2) and `DGEMM` (Level 3) and LU factorization based on BLAS Level 3 on an AMD Athlon 1200 Mhz processor (BLAS-routine implementation comes from ATLAS 3.2.1 [32,33]).

ceases to get larger as the system reaches its optimal behavior for a given routine. Other linear algebra, including LINPACK's LU factorization, shown for comparison in Figure 2, have their asymptotic rates as well.

To guard against excessive optimization encroaching upon the correctness of the solution, constraints are imposed on the set-up of the matrix and the numerical properties of the solution. This is achieved by requiring the use of the driver code from LINPACK 100 which generates random matrix entries, calls the routines to solve the problem (these may be replaced by the user), verifies that the answer is correct and computes the total number of operations (independently of the method) as $2n^3/3 + 2n^2$ (where $n = 1000$). The answer is correct if it has the same relative accuracy as standard techniques, such as Gaussian elimination used in the LINPACK package. By relative accuracy we mean that the scaled residual is a slowly growing function of matrix dimension $n$. For completeness, we only mention that this is achieved through the following standard result which holds regardless of the conditioning of $A$:

$$\frac{\|Ax - b\|}{\|A\| \cdot \|x\| \cdot n \cdot \varepsilon} = O(1)$$

Copyright © 2003 John Wiley & Sons, Ltd.


15320634, 2003, 9, Downloaded from https://onlinelibrary.wiley.com/doi/10.1002/cpe.728 by University Of Maryland, Wiley Online Library on [01/11/2023]. See the Terms and Conditions (https://onlinelibrary.wiley.com/terms-and-conditions) on Wiley Online Library for rules of use; OA articles are governed by the applicable Creative Commons License

where $A \in \mathbb{R}^{n\times n}$, $x, b \in \mathbb{R}^n$, $\varepsilon$ is the machine precision for 64-bit floating point arithmetic

$$\varepsilon = \max_{x>0} \text{float}(1+x) = 1$$

(float($x$) is machine representation of $x$) and $\|\cdot\|$ is any consistent matrix and vector norm.

With the arrival of parallel computers yet another requirement of the LINPACK benchmark had to be reconsidered. The so-called HPLinpack Benchmark allows for matrix dimension $n$ to be made as large as necessary so that asymptotic performance can be achieved. The following quantities are reported for each system:

- $R_{\max}$, the performance in Gflop s$^{-1}$ for the largest problem run on a machine;
- $N_{\max}$, the size of the largest problem run on a machine;
- $N_{1/2}$, the size where half the $R_{\max}$ execution rate is achieved;
- $R_{\text{peak}}$, the theoretical peak performance Gflop s$^{-1}$ for the machine.

To summarize, the rules for HPLinpack are: solve systems of equations by some method allowing the problem size $n$ to vary and measure the execution time for each problem size. In computing the floating-point execution rate, use $2n^3/3 + 2n^2$ operations independent of the actual method used (if Gaussian elimination is chosen then partial pivoting must be used), compute and report a residual for the accuracy of solution as $\|Ax - b\|/(\|A\|\|x\|)$.

## 5. THE TOP500 LIST

Statistics on high-performance computers are of major interest to manufacturers and potential users. They wish to know not only the number of systems installed, but also the location of the various supercomputers within the high-performance computing community and the applications for which a computer system is being used. Such statistics can facilitate the establishment of collaborations, the exchange of data and software and provide a better understanding of the high-performance computer market.

Statistical lists of supercomputers are not new. Every year since 1986, Hans Meuer [34] has published system counts of the major vector computer manufacturers, based principally on those at the Mannheim Supercomputer Seminar. However, statistics based merely on the name of the manufacturer are no longer useful. New statistics are required that reflect the diversification of supercomputers, the enormous performance difference between low-end and high-end models, the increasing availability of massively parallel processing (MPP) systems and the strong increase in computing power of the high-end models of workstations such as symmetric multiprocessors (SMP).

To provide this new statistical foundation, the TOP500 list was created in 1993 to assemble and maintain a list of the 500 most powerful computer systems. Its first edition was partially based on statistical lists published by other parties for different purposes [35,36], while today it relies on submissions from computer system users and vendors. The list is compiled twice a year with the help of high-performance computer experts, computational scientists, manufacturers and the Internet community in general.

It is true that, in the list, computers are ranked by their performance on the HPLinpack Benchmark. However, in an attempt to obtain uniformity across all computers in performance reporting, the

Copyright © 2003 John Wiley & Sons, Ltd.


15320634, 2003, 9, Downloaded from https://onlinelibrary.wiley.com/doi/10.1002/cpe.728 by University Of Maryland, Wiley Online Library on [01/11/2023]. See the Terms and Conditions (https://onlinelibrary.wiley.com/terms-and-conditions) on Wiley Online Library for rules of use; OA articles are governed by the applicable Creative Commons License



algorithm used in solving the system of linear equations in the benchmark routine must conform to the standard operation count for LU factorization with partial pivoting. In particular, the operation count for the algorithm must be $2/3n^3 + O(n^2)$ floating point operations. This excludes the use of a fast matrix multiply algorithm like Strassen's [35–38] or Coppersmith and Winograd's [41] methods for matrix multiplication. Even though there is no specific requirement for the method used for measuring performance, there exists a reference implementation of the benchmark called HPL [42]. While only meant as a guideline, it is being widely used to provide data for the TOP500 list, since it uses external routines for matrix–matrix operations which are supplied by the vendor and are very well tuned for a given computer system. A detailed description of HPL is provided in the following section. As closing remarks, we would like to mention other software packages and technologies that can be used for TOP500 submission. They include: HPF [43,44], PESSL [45], PLAPACK [46], ScaLAPACK [47] or even LAPACK [30] combined with either OMP [48] or `pthreads` [49].

They provide a Reference Implementation, even though the docs force people to use it.

## 6. HPL

This section gives a rather detailed description of the HPL code. However, to limit the size of this exposition, a substantial amount of technical information is omitted for which the reader is referred to the supplied references.

### 6.1. Overview

HPL is a portable implementation of the HPLinpack Benchmark written in C. At the same time, it can be regarded as a software package that generates, solves, checks and times the solution process of a random dense linear system of equations on distributed-memory computers. The package uses 64-bit floating-point arithmetic and portable routines for linear algebra operations and message passing. The former ones can either be BLAS or Vector Signal Image Processing Library (VSIPL) [50] while the latter are from MPI [51,52]. The true advantage of HPL is the fact that it allows the selection of multiple factorization algorithms. Figure 3 shows an outline of HPL's driver code, which is modeled after the original LINPACK 100 benchmark code.

### 6.2. Algorithm

HPL generates and solves a linear system of equations of order $n$:

$$Ax = b; \quad A \in \mathbb{R}^{n\times n}; \quad x, b \in \mathbb{R}^n$$

by first computing LU factorization with row partial pivoting of the $n$ by $n+1$ coefficient matrix $[A, b]$:

$$P_r[A, b] = [[LU], y], \quad P_r, L, U \in \mathbb{R}^{n\times n}, \quad y \in \mathbb{R}^n$$

Since the row pivoting (represented by the permutation matrix $P_r$) and the lower triangular factor $L$ are applied to $b$ as the factorization progresses, the solution $x$ is obtained in one step by solving the upper triangular system

$$Ux = y$$

The lower triangular matrix $L$ is left unpivoted and the array of pivots is not returned.

Copyright © 2003 John Wiley & Sons, Ltd.


15320634, 2003, 9, Downloaded from https://onlinelibrary.wiley.com/doi/10.1002/cpe.728 by University Of Maryland, Wiley Online Library on [01/11/2023]. See the Terms and Conditions (https://onlinelibrary.wiley.com/terms-and-conditions) on Wiley Online Library for rules of use; OA articles are governed by the applicable Creative Commons License

```
/* Generate and partition matrix data among MPI computing nodes. */
/* ... */

MPI_Barrier(...); /* All the nodes start at the same time. */

HPL_ptimer(...);  /* Start wall-clock timer. */

HPL_pdgesv(...);  /* Solve system of equations. */

HPL_ptimer(...);  /* Stop  wall-clock timer. */

MPI_Reduce(...);  /* Obtain the maximum wall-clock time. */

/* Gather statistics about performance rate (base on the maximum wall-clock time)
 and accuracy of the solution. */
/* ... */
```

Figure 3. Main computational steps performed by HPL to obtain the HPLinpack Benchmark rating.

| | | | |
|---|---|---|---|
| $P_0$ | $P_1$ | $P_0$ | $P_1$ |
| $P_2$ | $P_3$ | $P_2$ | $P_3$ |
| $P_0$ | $P_1$ | $P_0$ | $P_1$ |
| $P_2$ | $P_3$ | $P_2$ | $P_3$ |

Figure 4. Ownership of dense subblocks in the two-dimensional block-cyclic data distribution used by HPL. The number of processors is four (named $P_0$, $P_1$, $P_2$ and $P_3$), they are organized in a $2 \times 2$ grid ($P = Q = 2$). The number of subblocks is four in both dimensions ($n/n_B = 4$).

Figure 4 shows the two-dimensional block-cyclic data distribution used by HPL. The data is distributed onto a two-dimensional grid (of dimensions $P \times Q$) of processes according to the block-cyclic scheme to ensure good load balance as well as the scalability of the algorithm. The $n \times (n + 1)$ coefficient matrix is logically partitioned into blocks (each of dimension $n_B \times n_B$; where $n_B$ is referred to as *blocking factor*), that are cyclically dealt onto the $P \times Q$ process grid. This is done in both dimensions of the matrix.

The right-looking variant [9] has been chosen for the main loop of the LU factorization. This computation is logically partitioned with the same block size $n_B$ that was used for the data distribution. This means that, at each iteration of the loop, a panel of $n_B$ columns is factored and the trailing submatrix is updated.

At a given iteration of the main loop and because of the Cartesian property of the distribution scheme, each panel factorization occurs in one column of processes. This particular part of the computation lies on the critical path of the overall algorithm. For this operation, the user is offered a choice of

Copyright © 2003 John Wiley & Sons, Ltd.


15320634, 2003, 9, Downloaded from https://onlinelibrary.wiley.com/doi/10.1002/cpe.728 by University Of Maryland, Wiley Online Library on [01/11/2023]. See the Terms and Conditions (https://onlinelibrary.wiley.com/terms-and-conditions) on Wiley Online Library for rules of use; OA articles are governed by the applicable Creative Commons License



three (Crout, left- and right-looking) recursive variants based on matrix–matrix multiply. The software also allows the user to choose how many sub-panels the current panel should be divided into at each recursion level. Furthermore, one can also select at run-time the recursion stopping criterion in terms of the number of columns left to factor. When this threshold is reached, the sub-panel will then be factored using one of the three (Crout, left- or right-looking) factorization algorithms based on matrix–vector operations. Finally, for each panel's column, the pivot search and the associated swap and broadcast operations of the pivot row are combined into one single communication step. A binary-exchange (leave-on-all) reduction performs these three operations at once, which reduces the number of message exchanges but requires the high-performance network to perform well.

Once the panel factorization has been performed, the factored panel of columns is broadcast to the other process columns. There are many possible broadcast algorithms and the software currently offers the following variants:

- increasing ring;
- modified increasing ring;
- increasing two-ring;
- modified increasing two-ring;
- bandwidth-reducing;
- modified bandwidth-reducing.

The modified variants relieve the next processor (the one that would participate in factorization of the panel after the current one) from the burden of sending messages (otherwise it has to receive as well as send matrix update data). The ring variants propagate the update data in a single pipeline fashion, whereas the two-ring variants propagate data in two pipelines concurrently. The bandwidth-reducing variants [36, 51–54] divide the message to be sent into a number of pieces and scatter it across a single row of the grid of processors, so that more messages are exchanged but the total volume of communication is independent of the number of processors. This becomes particularly important when the computing nodes are much faster relative to the interconnect.

Once the current panel has been broadcast (or during the broadcast operation) the trailing submatrix has to be updated. As mentioned before, the panel factorization lies on the critical path. This means that when the $k$th panel has been factored and then broadcast, the next most urgent task to complete is factorization and broadcast of panel $k + 1$. This technique is often referred to as a *look-ahead* (or *send-ahead*) in the literature. HPL allows the selection of various depths of look-ahead. By convention, a depth of zero corresponds to having no look-ahead, in which case the trailing submatrix is updated by the panel currently broadcast. Look-ahead consumes some extra memory to keep all the panels of columns currently in the look-ahead pipe. Our experimental results show that a look-ahead of depth one or two is most likely to achieve the best performance gain.

The update of the trailing submatrix by the last panel in the look-ahead pipe is performed in three phases. First, the pivots must be applied to form the current row panel of $U$. Second, upper triangular solve using the column panel occurs. Finally, the updated part of $U$ needs to be broadcast to each process within a single column so that the local rank update of size $n_B$ can take place. It has been decided to combine the swapping and broadcast of $U$ at the cost of replicating the solve. HPL provides two algorithms for this communication operation: one is based on the binary-exchange algorithm and the second on bandwidth-reducing techniques. The former variant is a modified leave-on-all reduction operation. The latter has communication volume complexity that solely depends on the size of $U$

Copyright © 2003 John Wiley & Sons, Ltd.


15320634, 2003, 9, Downloaded from https://onlinelibrary.wiley.com/doi/10.1002/cpe.728 by University Of Maryland, Wiley Online Library on [01/11/2023]. See the Terms and Conditions (https://onlinelibrary.wiley.com/terms-and-conditions) on Wiley Online Library for rules of use; OA articles are governed by the applicable Creative Commons License

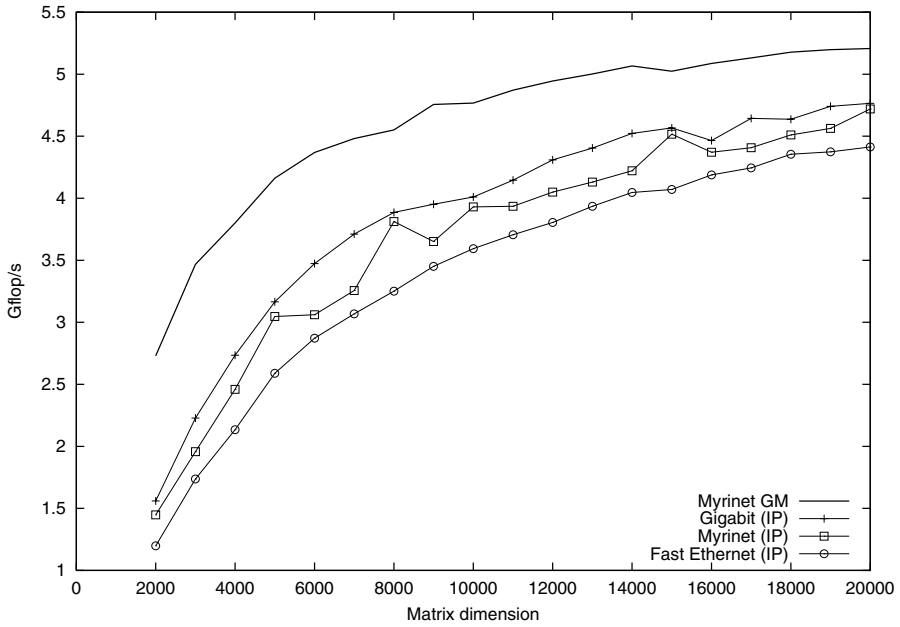



Figure 5. Comparison of HPL performance with three different interconnects, gigabit Ethernet, Myrinet and Fast Ethernet, on top of IP and Myrinet's native layer GM. The process grid consist of $4 \times 4$ Pentium III 550 MHz CPUs.

(the number of process rows only impacts the number of messages being exchanged) and, consequently, should outperform the previous variant for large problems on large machine configurations. In addition, both of the previous variants may be combined in a single run of the code.

After the factorization has ended, the backward substitution remains to be done. HPL uses look-ahead of depth one to do this. The right-hand side is forwarded in process rows in a decreasing-ring fashion, so that we solve $Qn_B$ entries at a time. At each step, this shrinking piece of the right-hand side is updated. The process just above the one owning the current diagonal block of the matrix updates its last $n_B$ entries of vector $x$, forwards it to the previous process column and then broadcasts it in the process column in a decreasing-ring fashion. The solution is then updated and sent to the previous process column. The solution of the linear system is left replicated in every process row.

To verify the result, the input matrix and right-hand side are regenerated. The following scaled residuals are computed ($\varepsilon$ is the relative machine precision):

$$r_n = \frac{\|Ax - b\|_\infty}{\|A\|_1 n\varepsilon}$$

Copyright © 2003 John Wiley & Sons, Ltd.


15320634, 2003, 9, Downloaded from https://onlinelibrary.wiley.com/doi/10.1002/cpe.728 by University Of Maryland, Wiley Online Library on [01/11/2023]. See the Terms and Conditions (https://onlinelibrary.wiley.com/terms-and-conditions) on Wiley Online Library for rules of use; OA articles are governed by the applicable Creative Commons License



Table IV. Description of the Compaq cluster used in tests.

| | |
|---|---|
| CPU | EV67 667 MHz |
| OS | True64 ver. 5 |
| C compiler | `cc` ver. 6.1 |
| C flags | `-arch host -tune host -std -O5` |
| MPI | native (linker flags: `-lmpi -lelan`) |
| BLAS | CXML |
| Date | September 2000 |

Table V. Performance (in Gflop $s^{-1}$) of HPL on a Compaq cluster with 64 computing nodes.

| | Matrix dimension | | | |
|---|---|---|---|---|
| Processor grid dimension | 5000 | 10 000 | 25 000 | 53 000 |
| $8 \times 8$ | 26.37 | 45.00 | 60.99 | 67.50 |

$$r_1 = \frac{\|Ax - b\|_\infty}{\|A\|_1 \|x\|_1 \varepsilon}$$
$$r_\infty = \frac{\|Ax - b\|_\infty}{\|A\|_\infty \|x\|_\infty \varepsilon}$$

A solution is considered numerically correct when all of these quantities are of order $O(1)$.

### 6.3. Performance results

The first set of performance results are shown in Figure 5. As in Figure 2, they reveal the asymptotic behavior of performance on a cluster of Pentium III 550 MHz processors. In addition, the figure shows how differences between messaging software (Myrinet GM and TCP/IP stack) and hardware (Myrinet, Gigabit Ethernet and Fast Ethernet) influence benchmarking results.

Performance tests were also performed on a larger system—a Compaq cluster installed at Oak Ridge National Laboratory in Tennessee, U.S.A. This cluster is listed at position 90 on the June 2001 TOP500 list. Its description and performance are given in Tables IV and V. The LINPACK Benchmark numbers for this system are presented in Table VI. All of the results, including the TOP500 submission for this system, were obtained with HPL. HPL was also used to benchmark the IBM system at Oak Ridge National Laboratory which is listed at position 8 on the June 2002 TOP500. The system has 864 IBM Power4 processors. The performance data of this processor is given in Table VII.

Copyright © 2003 John Wiley & Sons, Ltd.


15320634, 2003, 9, Downloaded from https://onlinelibrary.wiley.com/doi/10.1002/cpe.728 by University Of Maryland, Wiley Online Library on [01/11/2023]. See the Terms and Conditions (https://onlinelibrary.wiley.com/terms-and-conditions) on Wiley Online Library for rules of use; OA articles are governed by the applicable Creative Commons License

Table VI. LINPACK Benchmark numbers for the Compaq cluster obtained using HPL.

| CPUs/nodes | $N_{1/2}$ | $N_{max}$ | $R_{max}$ (Gflop s$^{-1}$) | $E$ (%) |
|---|---|---|---|---|
| 1/1 | 150 | 6625 | 1.14 | 100 |
| 4/1 | 800 | 12 350 | 4.36 | 95.6 |
| 16/4 | 2300 | 26 500 | 17.0 | 93.2 |
| 64/16 | 5700 | 53 000 | 67.5 | 92.5 |
| 256/64 | 14 000 | 106 000 | 263.6 | 90.1 |

Table VII. Hardware and performance characteristics of two families of processors from two different vendors.

| | Intel | | | | IBM | |
|---|---|---|---|---|---|---|
| Operation | Pentium III | Pentium III | P4 | P4 | Power3 | Power4 |
| Clock (MHz) | 550 | 933 | 1700 | 2530 | 375 | 1300 |
| L1 cache (I+D) (KB) | 16+16 | 16+16 | 12+8 | 12+8 | 32+64 | 32+64 |
| System bus (MHz) | 100 | 133 | 400 | 533 | 100 | 333 |
| Peak performance (Mflop s$^{-1}$) | 550 | 933 | 3400 | 5060 | 1500 | 5200 |
| DGEMM performance (%) | 73 | 71 | 64 | 69 | 86 | 67 |
| LINPACK 1000 (%) | 59 | 57 | 41 | 46 | 80 | 55 |
| HPLinpack (%) | 58 | 56 | N/A | N/A | 58 | 51 |

## 7. THE FUTURE OF THE BENCHMARK

Rather than embarking in risky fortune-telling, we will describe recent trends in high-performance computing which, we believe, will shape the near future of the LINPACK Benchmark suite.

One such trend is the ongoing improvement of CPU hardware which results in making Moore's law [57] reality. However, it is interesting to observe how the peak performance race (marketed to the public in the form of ever increasing clock rates) leaves behind much more practical metrics such as LINPACK numbers. Table VII shows evolution of characteristics of the key hardware components and performance numbers of two widely used families of superscalar processors. The performance numbers were given as percentages of the peak to show the decreasing trend of achievable computing power. Partially, it can be attributed to an insufficient amount of level 1 cache, which cannot provide enough buffering for the increased clock rates of the CPU and the system bus. Also, the tuning of vendor libraries lags behind the hardware development. The latter can be alleviated by self-tuning libraries such as ATLAS [32,33] and so we are most likely to see proliferation of such 'intelligent software' which encapsulates years of experience of experts and automates the deployment efforts of existing software to new hardware.

Copyright © 2003 John Wiley & Sons, Ltd.


15320634, 2003, 9, Downloaded from https://onlinelibrary.wiley.com/doi/10.1002/cpe.728 by University Of Maryland, Wiley Online Library on [01/11/2023]. See the Terms and Conditions (https://onlinelibrary.wiley.com/terms-and-conditions) on Wiley Online Library for rules of use; OA articles are governed by the applicable Creative Commons License



**ACKNOWLEDGEMENTS**

The authors acknowledge the use of the Oak Ridge National Laboratory Compaq cluster, funded by the Department of Energy's Office of Science and Energy Efficiency programs and the IBM supercomputer based on the Power4 microprocessor provided as a part of the Department of Energy's Scientific Discovery through Advanced Computing (SCiDAC) program.

This work was sponsored in part by the Applied Mathematical Sciences subprogram of the Office of Energy Research, U.S. Department of Energy, under Contract DE-AC05-96OR22464, and in part by the University of Tennessee.

**REFERENCES**

1. The LINPACK 1000x1000 benchmark program. (See http://www.netlib.org/benchmark/1000d for source code.).
2. Dongarra JJ, Bunch J, Moler C, Stewart GW. *LINPACK User's Guide*. SIAM: Philadelphia, PA, 1979.
3. Lawson C, Hanson R, Kincaid D, Krogh F. Basic Linear Algebra Subprograms for Fortran usage. *ACM Transactions on Mathematical Software* 1979; **5**:308–323.
4. Dongarra JJ. Performance of various computers using standard linear equations software. *Technical Report CS-89-85*, University of Tennessee, 2002.
   (An updated version of this report can be found at http://www.netlib.org/benchmark/performance.ps).
5. Pager D. Some notes on speeding up certain loops by software, firmware, and hardware means. *IEEE Transactions on Computers* 1972; 97–100.
6. Dongarra JJ, Hinds A. Unrolling loops in Fortran. *Software—Practice and Experience* 1979; **9**:219–226.
7. Knuth D. An empirical study of Fortran programs. *Software—Practice and Experience* 1971; **1**:105–133.
8. Dongarra JJ, Du Croz J, Hammarling S, Hanson R. An extended set of FORTRAN Basic Linear Algebra Subprograms. *ACM Transactions on Mathematical Software* 1988; **14**:1–17.
9. Dongarra JJ, Duff IS, Sorensen DC, van der Vorst HA. *Numerical Linear Algebra for High-Performance Computers*. SIAM: Philadelphia, PA, 1998.
10. Dongarra JJ, Hewitt T. Implementing dense linear algebra algorithms using mutlitasking on CRAY X-MP-4. *SIAM Journal of Science and Statistics in Computing* 1986; **7**(1):347–350.
11. Dongarra JJ, Sorensen DC. Linear algebra on high-performance computers. *Proceedings Parallel Computing 85*, Schendel U (ed.). North Holland, 1986; 3–32.
12. Dongarra JJ, Du Croz J, Duff IS, Hammarling S. A set of Level 3 FORTRAN Basic Linear Algebra Subprograms. *ACM Transactions on Mathematical Software* 1990; **16**:1–17.
13. Barron DW, Swinnerton-Dyer HPF. Solution of simultaneous linear equations using a magnetic tape store. *Computer J.* 1990; **3**:28–33.
14. Calahan DA. Block-oriented local-memory-based linear equation solution on the CRAY-2: Uniprocessor algorithms. *Proceedings International Conference on Parallel Processing*, August 1986, Schendel U (ed). IEEE Computer Society Press, 1986; 375–378.
15. Chartres B. Adaption of the Jacobi and Givens methods for a computer with magnetic tape backup store. *Technical Report 8*, University of Sydney, 1960.
16. Dave AK, Duff IS. Sparse matrix calculations on the CRAY-2. *Technical Report CSS 197*, AERE Harwell, 1986.
17. DuCroz J, Nugent S, Reid J, Taylor D. Solving large full sets of linear equations in a paged virtual store. *ACM Transactions on Mathematical Software* 1981; **7**(4):527–536.
18. McKellar AC, Coffman EG Jr. Organizing matrices and matrix operations for paged memory systems. *Communications of the ACM* 1969; **12**(3):153–165.
19. Berry M, Gallivan K, Harrod W, Jalby W, Lo S, Meier U, Philippe B, Sameh A. Parallel algorithms on CEDAR system. *Technical Report Report No. 581*, CSRD, 1986.
20. Bischof C, Van Loan CF. The WY representation for products of Householder matrices. *SIAM SISSC* 1987; **8**(2).
21. Bucher I, Jordan T. Linear algebra programs for use on a vector computer with a secondary solid state storage device. *Advances in Computer Methods for Practical Differential Equations*, Vichnevetsky R, Stepleman R (eds). IMACS, 1984; 546–550.
22. IBM. *Engineering and Scientific Subroutine Library*. IBM, 1986. Program Number: 5668-863.
23. Robert Y, Suguazerro P. The LU decomposition algorithm and its efficient Fortran implementation on the IBM 3090 vector multiprocessor. *Technical Report ECSEC Report ICE-0006*, IBM, March 1987.

Copyright © 2003 John Wiley & Sons, Ltd. 

15320634, 2003, 9, Downloaded from https://onlinelibrary.wiley.com/doi/10.1002/cpe.728 by University Of Maryland, Wiley Online Library on [01/11/2023]. See the Terms and Conditions (https://onlinelibrary.wiley.com/terms-and-conditions) on Wiley Online Library for rules of use; OA articles are governed by the applicable Creative Commons License

24. Schreiber R. *Engineering and Scientific Subroutine Library. Module Design Specification*. SAXPY Computer Corporation, 255 San Geronimo Way, Sunnyvale, CA 94086, 1986.
25. Dongarra JJ, Eijkhout V, Luszczek P. Recursive approach in sparse matrix LU factorization. *Scientific Programming* 2001; **9**(1):51–60.
26. Duff IS. Full matrix techniques in sparse Gaussian elimination. *Numerical Analysis Proceedings*, Dundee, 1981 (*Lecture Notes in Mathematics*, vol. 912). Springer: Berlin, 1981; 71–84.
27. George A, Rashwan H. Auxiliary storage methods for solving finite element systems. *SIAM SISSC* 1985; **6**:882–910.
28. Gustavson FG. Recursion leads to automatic variable blocking for dense linear-algebra algorithms. *IBM Journal of Research and Development* 1997; **41**(6):737–755.
29. Toledo S. Locality of reference in LU decomposition with partial pivoting. *SIAM Journal on Matrix Analysis and Applications* 1997; **18**(4).
30. Anderson E, Bai Z, Bischof C, Blackford SL, Demmel JW, Dongarra JJ, Du Croz J, Greenbaum A, Hammarling S, McKenney A, Sorensen DC. *LAPACK User's Guide* (3rd edn). Society for Industrial and Applied Mathematics: Philadelphia, PA, 1999.
31. Toledo S, Gustavson FG. The design and implementation of SOLAR, a portable library of rscalable out-of-core linear algebra computations. *Proceedings of the 4th Annual Workshop on I/O in Parallel and Distributed Systems*, May 1996. ACM Press, 1996; 28–40.
32. Dongarra JJ, Petitet A, Whaley RC. Automated empirical optimization of software and the ATLAS project. *Parallel Computing* 2001; **27**(1-2):3–25.
33. Dongarra JJ, Whaley RC. Automatically tuned linear algebra software (ATLAS). *Proceedings SC'89 Conference*. IEEE, 1998.
34. Meuer HW, Strohmaier E, Dongarra JJ, Simon HD. *Top500 Supercomputer Sites*, 17th edition, November 2 2001. (The report can be downloaded from http://www.netlib.org/benchmark/top500.html).
35. Ahrendt G. Weekly postings to comp.sys.super [1993].
36. Kahaner D. Kahaner report on supercomputer in Japan. *Technical Report*, The Computer Science Department, University of Arizona, 1992. ftp://ftp.cs.arizona.edu/japan/kahaner.reports/jsuper.92.
37. Bailey D, Lee K, Simon H. Using Strassen's algorithm to accelerate the solution of linear systems. *Journal of Supercomputing* 1990; **4**:357–371.
38. Grayson B, van de Geijn R. A high performance parallel Strassen implementation. *Parallel Processing Letters* 1996; **6**(1):3–12.
39. Paprzycki M, Cyphers C. Using Strassen's matrix multiplication in high performance solution of linear systems. *Computers and Mathematics with Applications* 1996; **31**(4/5):55–61.
40. Strassen V. Gaussian elimination is not optimal. *Numerical Mathematics* 1969; **13**:354–356.
41. Coppersmith D, Winograd S. Matrix multiplication via arithmetic progressions. *Journal of Symbolic Computation* 1990; **9**:251–280.
42. Petitet A, Whaley RC, Dongarra JJ, Cleary A. *HPL—A Portable Implementation of the High-Performance Linpack Benchmark for Distributed-Memory Computers*. Innovative Computing Laboratory, September 2000. Available at http://icl.cs.utk.edu/hpl/ and http://www.netlib.org/benchmark/hpl/.
43. High Performance Fortran Forum. High Performance Fortran Language specification. version 1.1. *Technical Report*, Rice University, November 1994.
44. High Performance Fortran Forum. High Performance Fortran Language specification. version 2.0. *Technical Report*, Rice University, January 1997.
45. IBM. *Parallel Engineering and Scientific Subroutine Library for AIX Version 2 Release 3*. IBM, 2001.
46. van de Geijn RA. *Using PLAPACK*. The MIT Press, 1997.
47. Blackford LS, Choi J, Cleary A, D'Azevedo E, Demmel JW, Dhillon IS, Dongarra JJ, Hammarling S, Henry G, Petitet A, Stanley K, Walker DW, Whaley RC. *ScaLAPACK Users' Guide*. Society for Industrial and Applied Mathematics: Philadelphia, PA, 1997.
48. Openmp: Simple, portable, scalable smp programming. http://www.openmp.org/.
49. International Organization for Standardization. *Information technology—Portable operating system interface (POSIX)—Part 1: System Application Programming Interface (API) [C language]*. ISO/IEC 9945-1:1996, Geneva, Switzerland, 1996.
50. Shwartz DA, Judd RR, Harrod WJ, Manley DP. VSIPL 1.02 API. http://www.vsipl.org/ [26 February 2002].
51. Message Passing Interface Forum. MPI: A Message-Passing Interface Standard (version 1.1). http://www.mpi-forum.org/ [1995].
52. Message Passing Interface Forum. MPI-2: Extensions to the Message-Passing Interface. http://www.mpi-forum.org/ [July 1997].
53. Chtchelkanova A, Gunnels J, Morrow G, Overfelt J, van de Geijn R. Parallel implementation of BLAS: General techniques for Level 3 BLAS. *Concurrency: Practice and Experience* 1997; **9**(9):837–857.

Copyright © 2003 John Wiley & Sons, Ltd. 

15320634, 2003, 9, Downloaded from https://onlinelibrary.wiley.com/doi/10.1002/cpe.728 by University Of Maryland, Wiley Online Library on [01/11/2023]. See the Terms and Conditions (https://onlinelibrary.wiley.com/terms-and-conditions) on Wiley Online Library for rules of use; OA articles are governed by the applicable Creative Commons License



54. Dongarra JJ, van de Geijn R, Walker DW. Scalability issues in the design of a library for dense linear algebra. *Journal of Parallel and Distributed Computing* 1994; **22**(3):523–537. (Also LAPACK Working Note No. 43).
55. van de Geijn R. Massively parallel LINPACK Benchmark on the Intel Touchstone DELTA and iPSC/860 systems. *1991 Annual Users Conference Proceedings*, Dallas, Texas. Intel Supercomputer Users Group, 1991.
56. van de Geijn R, Watts J. SUMMA: Scalable universal matrix multiplication algorithm. *Concurrency: Practice and Experience* 1997; **9**(4):255–274.
57. Moore GE. Cramming more components onto integrated circuits. *Electronics* 1965; **38**(8).

Copyright © 2003 John Wiley & Sons, Ltd. 

15320634, 2003, 9, Downloaded from https://onlinelibrary.wiley.com/doi/10.1002/cpe.728 by University Of Maryland, Wiley Online Library on [01/11/2023]. See the Terms and Conditions (https://onlinelibrary.wiley.com/terms-and-conditions) on Wiley Online Library for rules of use; OA articles are governed by the applicable Creative Commons License